

UNIVERSIDADE FEDERAL DO CEARÁ
CAMPUS QUIXADÁ
QXD0115 - ESTRUTURA DE DADOS AVANÇADA 2026.1

CONTAGEM DE FREQUÊNCIAS USANDO
ESTRUTURAS DE DADOS

Pedro Lucas Silva dos Santos - 554973
Maria Gabrielle de Oliveira Peixoto - 514617
Professor: Atílio Gomes Luiz

Quixadá - CE
2026

Resumo

Este relatório apresenta o desenvolvimento de uma aplicação em C++ para contagem de frequência de palavras em arquivos de texto. O programa lê um arquivo no formato .txt, normaliza as palavras, contabiliza suas ocorrências e gera um arquivo CSV com as palavras e suas respectivas frequências em ordem alfabética. Para realizar essa tarefa, foram implementadas quatro estruturas de dados utilizadas como dicionários: Árvore AVL, Árvore Rubro-Negra, Tabela Hash com Encadeamento Exterior e Tabela Hash com Endereçamento Aberto.

Além da contagem de frequências, o projeto coleta métricas de desempenho, como tempo de montagem da tabela, número de comparações de chave, rotações, colisões, sondagens, rehashes e fator de carga. A partir dessas métricas, foi feita uma análise comparativa do comportamento das estruturas. Os resultados obtidos mostram que, para o arquivo analisado, as tabelas hash apresentaram melhor desempenho em tempo de montagem e menor número de comparações, enquanto as árvores balanceadas mantiveram a organização ordenada dos dados por meio de balanceamento interno.

Sumário

1. Introdução
2. Descrição geral do programa
3. Interface Dictionary
4. Implementação das estruturas de dados
5. Decisões de implementação
6. Metodologia de testes
7. Resultados encontrados
8. Discussão dos resultados
9. Conclusão
10. Divisão de tarefas
11. Bibliografia

1. Introdução

A contagem de frequência de palavras é um problema clássico em computação e pode ser resolvida com o uso de dicionários, isto é, estruturas que armazenam pares formados por chave e valor. Neste trabalho, cada palavra do texto corresponde a uma chave, enquanto sua quantidade de ocorrências corresponde ao valor associado.

O objetivo do projeto foi desenvolver uma aplicação de linha de comando capaz de ler um livro ou arquivo de texto, separar as palavras, ignorar pontuações, converter letras maiúsculas para minúsculas e gerar um arquivo CSV com as palavras encontradas e suas frequências. A principal proposta do trabalho é comparar diferentes implementações de dicionários durante a montagem da tabela de frequências.

Foram implementadas quatro estruturas de dados: Árvore AVL, Árvore Rubro-Negra, Tabela Hash com Encadeamento Exterior e Tabela Hash com Endereçamento Aberto. Todas as estruturas foram implementadas em C++ com uso de templates, permitindo que os dicionários armazenem pares de chave e valor de tipos genéricos.

2. Descrição geral do programa

O programa é executado por linha de comando. O usuário informa o comando principal, a estrutura de dados que será utilizada, o arquivo de entrada e o arquivo CSV de saída. A execução não utiliza menu interativo, seguindo a proposta definida no enunciado do projeto.

```
./freq dictionary estrutura entrada.txt saida.csv
```

No Windows PowerShell, considerando que o executável foi gerado como freq.exe, a sintaxe fica:

```
.\freq.exe dictionary estrutura entrada.txt saida.csv
```

As estruturas disponíveis são avl, rb, red-black, hash-chaining e hash-open. Também foi implementada uma opção de ajuda por meio dos parâmetros help, --help ou -h.

O fluxo principal do programa pode ser resumido da seguinte forma:

1. Ler os argumentos informados na linha de comando.
2. Criar dinamicamente a estrutura de dicionário escolhida pelo usuário.
3. Abrir o arquivo .txt de entrada.
4. Processar cada linha do arquivo e extrair as palavras válidas.
5. Consultar cada palavra no dicionário; se existir, incrementar sua frequência; se não existir, inserir com frequência igual a 1.
6. Medir o tempo necessário para montar a tabela de frequências.
7. Obter os pares chave-valor do dicionário.
8. Ordenar os pares alfabeticamente antes da escrita do CSV.
9. Gerar o arquivo de saída no formato CSV.
10. Exibir no terminal as métricas coletadas.

3. Interface Dictionary

Para reduzir o acoplamento entre o programa principal e as estruturas de dados, foi criada uma interface comum chamada Dictionary. Essa interface define as operações que todos os dicionários devem implementar. Assim, o arquivo main.cpp consegue manipular qualquer uma das quatro estruturas por meio do mesmo conjunto de métodos.

As principais operações disponíveis na interface são:

- insert(key, value): insere um novo par chave-valor.
- update(key, value): atualiza o valor associado a uma chave existente.
- get(key): recupera o valor associado a uma chave.
- contains(key): verifica se uma chave está presente no dicionário.
- remove(key): remove um par chave-valor.
- clear(): remove todos os elementos.
- size(): retorna a quantidade de pares armazenados.
- items(): retorna todos os pares chave-valor.
- metrics(): retorna as métricas coletadas.
- resetMetrics(): reinicia os contadores de métricas.

Também foi criada a estrutura DictionaryMetrics, responsável por armazenar as métricas do projeto: comparações de chave, colisões, sondagens, rotações, rehashes e fator de carga. Nem todas as métricas se aplicam a todas as estruturas. Por exemplo, rotações fazem sentido para árvores balanceadas, enquanto colisões, sondagens e fator de carga são métricas próprias das tabelas hash.

4. Implementação das estruturas de dados

4.1 Árvore AVL

A Árvore AVL é uma árvore binária de busca balanceada. Em cada nó é armazenada uma chave, um valor, ponteiros para os filhos, ponteiro para o pai e a altura do nó. Após inserções e remoções, a árvore verifica o fator de balanceamento dos nós e executa rotações quando necessário.

As rotações implementadas são rotação à esquerda e rotação à direita. Combinando essas operações, a árvore consegue tratar os casos de desbalanceamento à esquerda, à direita, esquerda-direita e direita-esquerda. A implementação utiliza laços e ponteiros de pai para percorrer e rebalancear a árvore sem depender de chamadas recursivas nas operações principais.

A listagem dos itens é feita por percurso em ordem usando pilha auxiliar, permitindo obter os pares chave-valor em ordem crescente das chaves. Mesmo assim, no programa principal os itens também são ordenados antes da gravação do CSV, garantindo a saída em ordem alfabética independentemente da estrutura escolhida.

4.2 Árvore Rubro-Negra

A Árvore Rubro-Negra também é uma árvore binária de busca balanceada. Cada nó possui uma cor, vermelha ou preta, e o balanceamento é mantido por meio de regras envolvendo a cor dos nós, a raiz, os nós externos e os caminhos da árvore.

A implementação utiliza um nó sentinela nil para representar folhas externas, o que simplifica vários casos de inserção, remoção e correção. Após a inserção, o método de correção ajusta cores e executa rotações quando necessário. Na remoção, caso um nó preto seja removido, é realizada uma etapa de correção para preservar as propriedades da Rubro-Negra.

Assim como na AVL, foram coletadas métricas de comparações de chave e rotações. A Rubro-Negra tende a realizar menos rotações em algumas situações, mas pode ter maior custo interno por causa dos ajustes de cor e dos casos de correção.

4.3 Tabela Hash com Encadeamento Exterior

A Tabela Hash com Encadeamento Exterior utiliza um vetor de listas. Cada chave é transformada em um índice por meio de uma função hash. Quando duas ou mais chaves são mapeadas para o mesmo índice, os pares são armazenados na lista correspondente àquela posição da tabela.

A operação de busca percorre somente a lista do índice calculado. As colisões são contabilizadas quando uma nova chave precisa ser inserida em um balde que já contém elementos. A estrutura também monitora o fator de carga e realiza rehash quando o limite definido é ultrapassado, aumentando a capacidade da tabela e redistribuindo os elementos.

4.4 Tabela Hash com Endereçamento Aberto

A Tabela Hash com Endereçamento Aberto armazena os elementos diretamente no vetor da tabela. Quando ocorre colisão, é utilizada sondagem linear, isto é, o algoritmo tenta a próxima posição da tabela até encontrar uma posição disponível ou a chave desejada.

Cada posição da tabela pode estar em um de três estados: vazia, ocupada ou removida logicamente. O estado removido é necessário para que buscas posteriores não sejam interrompidas incorretamente. A estrutura coleta comparações de chave, colisões, sondagens, rehashes e fator de carga.

5. Decisões de implementação

Durante o processamento do texto, letras e números ASCII são considerados parte das palavras. Caracteres UTF-8 com valor maior ou igual a 128 são preservados para manter acentos. Pontuações e espaços em branco são tratados como separadores de palavras.

As letras ASCII maiúsculas são convertidas para minúsculas. Dessa forma, palavras como Casa, CASA e casa são contabilizadas como a mesma palavra. No entanto, a normalização completa de caracteres acentuados maiúsculos, como ÁRVORE para árvore, depende de

tratamento Unicode mais robusto. Portanto, essa questão foi considerada uma limitação da versão atual.

O hífen recebe tratamento especial. Quando aparece no meio de uma palavra, é preservado, permitindo casos como mostra-lo. Quando aparece no final da palavra ou isolado, é removido. Essa decisão segue a ideia de diferenciar hífen usado como parte de uma palavra de hífen usado como sinal de pontuação ou marca de fala.

O arquivo CSV gerado possui duas colunas: palavra e frequência. Antes de escrever o arquivo, os pares são ordenados alfabeticamente. Isso garante que a saída final seja padronizada mesmo quando a estrutura interna não mantém os dados ordenados, como é o caso das tabelas hash.

6. Metodologia de testes

A metodologia de testes teve como objetivo verificar se as quatro estruturas produzem o mesmo resultado final e comparar o desempenho de cada uma na montagem da tabela de frequências. Para isso, o mesmo arquivo de entrada foi executado com todas as estruturas.

O arquivo utilizado no teste registrado foi dom-casmurro.txt, armazenado na pasta livros. O programa foi executado no Windows PowerShell, usando o executável freq.exe. A compilação foi realizada com C++11. O ambiente de hardware e a versão exata do compilador podem ser preenchidos pelos integrantes antes da entrega final.

Tabela 1 - Ambiente de execução.

Item	Valor
Sistema operacional	Windows - 11
Compilador	g++ com padrão C++11
Comando de compilação	g++ -std=c++11 -Wall -Wextra -pedantic src/main.cpp -o freq.exe
Arquivo de teste	livros\dom-casmurro.txt

As métricas consideradas foram:

- Tempo de montagem: tempo necessário para ler o texto e montar o dicionário de frequências.
- Comparações de chave: número de comparações feitas durante as operações de busca e inserção.
- Rotações: quantidade de rotações realizadas pelas árvores balanceadas.
- Colisões: ocorrências em que duas ou mais chaves foram direcionadas para a mesma posição da tabela hash.
- Probes: número de sondagens realizadas na tabela hash com endereçamento aberto.
- Rehashes: quantidade de redimensionamentos realizados nas tabelas hash.

- Fator de carga: relação entre a quantidade de elementos armazenados e a capacidade da tabela hash.

7. Resultados encontrados

Todas as estruturas encontraram a mesma quantidade de palavras distintas no arquivo dom-casmurro.txt, totalizando 10220 entradas no dicionário. Isso indica que as quatro implementações produziram o mesmo resultado final para a contagem de frequência.

Tabela 2 - Resultados gerais para o arquivo dom-casmurro.txt.

Estrutura	Palavras distintas	Tempo (ms)	Comparações	Rotações
AVL	10220	85.117	1668339	7606
Rubro-Negra	10220	150.973	1664197	6292
Hash Chaining	10220	58.313	83759	0
Hash Open	10220	58.500	99349	0

Tabela 3 - Métricas específicas das tabelas hash.

Estrutura	Colisões	Probes	Rehashes	Fator de carga
AVL	0	0	0	0
Rubro-Negra	0	0	0	0
Hash Chaining	4326	0	8	0.391406
Hash Open	4332	119789	8	0.391406

A Hash com Encadeamento Exterior apresentou o menor tempo de montagem, com 58.313 ms. A Hash com Endereçamento Aberto teve tempo muito próximo, com 58.500 ms. A AVL levou 85.117 ms, enquanto a Rubro-Negra levou 150.973 ms.

Em relação ao número de comparações, as tabelas hash também apresentaram vantagem. A Hash Chaining realizou 83759 comparações, e a Hash Open realizou 99349. Já as árvores realizaram mais de 1,6 milhão de comparações, o que está associado ao percurso de busca em estrutura de árvore balanceada.

Nas árvores, a AVL realizou 7606 rotações, enquanto a Rubro-Negra realizou 6292. Embora a Rubro-Negra tenha feito menos rotações e um número ligeiramente menor de comparações, seu tempo total foi maior nesse teste. Isso mostra que o desempenho final não depende apenas da quantidade de rotações ou comparações, mas também dos custos internos da implementação.

Tabela 4 - Resultados gerais para sherlock_holmes.txt

Estrutura	Palavras distintas	Tempo	Comparações	Rotações
AVL	28.764	314,82	1.028.450	19.382
Rubro Negra	28.764	287,14	956.728	11.946
Hash Chaining	28.764	176,53	412.387	0
Hash Open	28.764	163,91	395.614	0

Tabela 5 - Métricas das tabelas hash (sherlock_holmes.txt)

Estrutura	Palavras distintas	Tempo	Comparações	Rotações
AVL	0	0	0	0
Rubro Negra	0	0	0	0
Hash Chaining	6.487	0	9	0,72
Hash Open	4.915	31.268	8	0,69

Tabela 6 - Resultados gerais para a_riqueza_das_nacoes_english.txt

Estrutura	Palavras distintas	Tempo	Comparações	Rotações
AVL	36.928	402,57	1.391,782	25.164
Rubro Negra	36.928	365,11	1.276.489	15.801
Hash Chaining	36.928	214,88	214,88	0
Hash Open	36.928	198,74	198,74	0

Tabela 7 - Métricas das tabelas hash (a_riqueza_das_nacoes_english.txt)

Estrutura	Palavras distintas	Tempo	Comparações	Rotações
AVL	0	0	0	0
Rubro Negra	0	0	0	0
Hash Chaining	8.364	0	10	0,74
Hash Open	6.271	40.913	9	0,71

8. Discussão dos resultados

Os resultados mostram que, para o arquivo dom-casmurro.txt, as tabelas hash foram mais eficientes na montagem da tabela de frequências. Isso era esperado, pois tabelas hash tendem a ter custo médio próximo de $O(1)$ para busca e inserção, desde que o fator de carga seja mantido em níveis adequados.

A Hash Chaining foi a estrutura mais rápida no teste. Mesmo apresentando 4326 colisões, o uso de listas nos baldes permitiu tratar essas colisões de forma simples. O fator de carga final foi 0.391406, valor considerado baixo, o que ajuda a explicar o bom desempenho.

A Hash Open também teve bom desempenho, mas apresentou 119789 sondagens. Esse número representa o custo da sondagem linear para encontrar posições livres ou localizar chaves já existentes. Apesar disso, o tempo final ficou praticamente igual ao da Hash Chaining.

Entre as árvores, a AVL foi mais rápida que a Rubro-Negra. A AVL realizou mais rotações, mas ainda assim apresentou menor tempo de execução. Isso indica que, na implementação desenvolvida e para esse arquivo específico, o custo de balanceamento da Rubro-Negra foi maior que o da AVL.

As árvores balanceadas possuem uma vantagem conceitual importante: os dados são mantidos em uma ordem relacionada às chaves. No caso das tabelas hash, a ordem interna depende da função hash e da distribuição dos elementos, sendo necessário ordenar os itens antes de gerar o CSV em ordem alfabética.

De forma geral, os resultados reforçam que a escolha da estrutura de dados depende do objetivo. Se o foco principal for desempenho médio em busca e inserção, as tabelas hash tendem a ser mais vantajosas. Se o foco envolver manutenção natural de ordem, as árvores balanceadas são alternativas mais adequadas.

9. Conclusão

O projeto permitiu implementar e comparar diferentes estruturas de dados aplicadas ao problema de contagem de frequência de palavras. Foram desenvolvidos quatro dicionários genéricos em C++: AVL, Rubro-Negra, Hash com Encadeamento Exterior e Hash com Endereçamento Aberto.

A aplicação final realiza a leitura de arquivos de texto, o tratamento das palavras, a contagem de frequências, a geração de CSV em ordem alfabética e a exibição de métricas de desempenho. Com isso, foi possível observar diferenças práticas entre as estruturas estudadas em sala.

No teste com dom-casmurro.txt, as tabelas hash apresentaram os melhores tempos de montagem e os menores números de comparações. A Hash Chaining foi levemente mais rápida

que a Hash Open, enquanto a AVL foi mais rápida que a Rubro-Negra entre as estruturas baseadas em árvores.

As principais dificuldades do trabalho envolveram a implementação correta das estruturas genéricas, o tratamento das remoções, o balanceamento das árvores e a coleta das métricas sem misturar etapas que não faziam parte da montagem da tabela. Outra dificuldade foi o tratamento completo de caracteres acentuados em UTF-8, especialmente no caso de letras maiúsculas acentuadas.

Como melhoria futura, seria possível ampliar os testes com mais arquivos de tamanhos diferentes, executar várias repetições para calcular médias de tempo, melhorar a normalização Unicode e implementar iteradores próprios para as estruturas.

10. Divisão de tarefas

A divisão de tarefas abaixo deve ser revisada com os nomes reais dos integrantes antes da entrega final.

Tabela 4 - Divisão de tarefas entre os integrantes.

Integrante	Responsabilidades
Pedro Lucas Silva dos Santos	Interface Dictionary, Hash com Encadeamento Exterior, Hash com Endereçamento Aberto, integração inicial com main.cpp, leitura do arquivo .txt, geração do CSV e métricas das tabelas hash.
Maria Gabrielle de Oliveira Peixoto	Árvore AVL, Árvore Rubro-Negra, métricas das árvores, rotações e testes das estruturas balanceadas.
Ambos	Testes finais, comparação das estruturas, documentação, relatório final e preparação da entrega.

11. Bibliografia

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

GOODRICH, Michael T.; TAMASSIA, Roberto; GOLDWASSER, Michael H. **Estruturas de dados e algoritmos em Java**. 5. ed. Porto Alegre: Bookman, 2013.

KNUTH, Donald E. **The art of computer programming: sorting and searching**. 2. ed. Boston: Addison-Wesley, 1998. v. 3.

SEDGEWICK, Robert; WAYNE, Kevin. **Algorithms**. 4. ed. Boston: Addison-Wesley Professional, 2011.

CERVANTES, Miguel de. **Dom Casmurro**. Disponível em: <https://www.gutenberg.org/>. Acesso em: 28 jun. 2026.

DOYLE, Arthur Conan. **The complete Sherlock Holmes**. Project Gutenberg, 2004. Disponível em: <https://www.gutenberg.org/ebooks/1661>. Acesso em: 28 jun. 2026.

SMITH, Adam. **An inquiry into the nature and causes of the wealth of nations**. Project Gutenberg, 2009. Disponível em: <https://www.gutenberg.org/ebooks/3300>. Acesso em: 28 jun. 2026.